

Short Course in MATLAB

Lecture 2

Amit Harlev

Demo: $1 + 1/4 + 1/9 + 1/16 + \dots$

How might we compute this sum in MATLAB? (up to, say, the 1,000,000th term)

```
1 total = 0;
2 for i = 1:1000000
3     total = total + 1/i^2;
4 end
5 disp(sqrt(6 * total)); % any guesses for what this is?
```

This will print the value 3.141591698660509.

Fun fact

$$1 + \frac{1}{4} + \frac{1}{9} + \frac{1}{16} + \dots = \frac{\pi^2}{6}.$$

Demo cont.

Can we see how quickly this approaches π ?

Demo cont.

Can we see how quickly this approaches π ?

We need to keep track of total after each iteration of the loop:

```
1 totals = [1];  
2 for i = 2:100  
3     totals(i) = totals(i-1) + 1/i^2;  
4 end
```

Note the new usage of the `( )` syntax: we are growing the array by accessing an index larger than the current size of the array.

Demo cont.

Can we see how quickly this approaches π ?

We need to keep track of total after each iteration of the loop:

```
1 totals = [1];  
2 for i = 2:100  
3     totals(i) = totals(i-1) + 1/i^2;  
4 end
```

Note the new usage of the `( )` syntax: we are growing the array by accessing an index larger than the current size of the array.

In MATLAB, one would usually *preallocate* the array because growing an array one element at a time is slow for the computer to do.

Demo cont.

Can we see how quickly this approaches π ?

We need to keep track of total after each iteration of the loop:

```
1 n = 100;
2 totals = zeros(1, n);
3 totals(1) = 1;
4 for i = 2:n
5     totals(i) = totals(i-1) + 1/i^2;
6 end
```

~~Note the new usage of the () syntax: we are growing the array by accessing an index larger than the current size of the array.~~

In MATLAB, one would usually *preallocate* the array because growing an array one element at a time is slow for the computer.

Demo cont.

Can we see how quickly this approaches π ?

```
1 n = 100;
2 totals = zeros(1, n);
3 totals(1) = 1;
4 for i = 2:n
5     totals(i) = totals(i-1) + 1/i^2;
6 end
7
8 pi_estimates = zeros(1, n);
9 for i = 1:n
10    pi_estimates(i) = sqrt(totals(i) * 6);
11 end
```

Demo cont.

Can we see how quickly this approaches π ?

```
1 n = 100;
2 totals = zeros(1, n);
3 totals(1) = 1;
4 for i = 2:n
5     totals(i) = totals(i-1) + 1/i^2;
6 end
7
8 pi_estimates = zeros(1, n);
9 for i = 1:n
10    pi_estimates(i) = sqrt(totals(i) * 6);
11 end
12
13 plot(pi_estimates);
14 yline(pi, '--');
```

More on plotting in MATLAB in the future!

Comparison Operators

Comparison operators compare two values and produce either `true` or `false`.

Operator	Meaning
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to
==	Equal to
~=	Not equal to

Logical Operators

Logical operators combine or negate conditions.

Operator	Meaning
<code>&&</code>	AND: are both conditions true?
<code> </code>	OR: is at least one condition true?
<code>~</code>	NOT: reverse true and false

Logical Operators

Logical operators combine or negate conditions.

Operator	Meaning
<code>&&</code>	AND: are both conditions true?
<code> </code>	OR: is at least one condition true?
<code>~</code>	NOT: reverse true and false

- `(x > 0) && (x < 10)` checks whether `x` is between 0 and 10.

Logical Operators

Logical operators combine or negate conditions.

Operator	Meaning
<code>&&</code>	AND: are both conditions true?
<code> </code>	OR: is at least one condition true?
<code>~</code>	NOT: reverse true and false

- `(x > 0) && (x < 10)` checks whether `x` is between 0 and 10.
- `(x < 0) || (x > 10)` checks whether `x` is outside that range.

Logical Operators

Logical operators combine or negate conditions.

Operator	Meaning
<code>&&</code>	AND: are both conditions true?
<code> </code>	OR: is at least one condition true?
<code>~</code>	NOT: reverse true and false

- `(x > 0) && (x < 10)` checks whether `x` is between 0 and 10.
- `(x < 0) || (x > 10)` checks whether `x` is outside that range.
- `~(x == 5)` checks whether `x` is not 5.

Conditional Statements

Conditional statements run code only when certain conditions are **true**.

```
1 if condition1
2     % Code if condition1 is true
3 elseif condition2
4     % Code if condition2 is true
5 else
6     % Code if neither is true
7 end
```

- MATLAB runs only the first branch whose condition is true.
- The **elseif** and **else** branches are optional.
- Can have any number of **elseif** branches.

Conditional Statement Example

```
1 temperature = 65;
2
3 if temperature < 50
4     disp("Bring a coat.")
5 elseif temperature < 70
6     disp("Bring a jacket.")
7 else
8     disp("No jacket needed.")
9 end
```

Since `temperature < 70` is true, MATLAB displays: Bring a jacket.

While Loops

A `while` loop repeats code as long as a condition is true.

```
1 count = 1;  
2  
3 while count <= 5  
4     disp(count);  
5     count = count + 1;  
6 end
```

- MATLAB checks the condition before each repetition.
- The loop stops when `count <= 5` becomes false.
- Something inside the loop must eventually make the condition false. Otherwise, the program creates an infinite loop.

While Loop Better Example

How many times do we need to roll a die before getting a 6?

```
1 rolls = 0;
2 result = 0;
3
4 while result ~= 6
5     result = randi(6); % generates random integer between 1 and 6
6     rolls = rolls + 1;
7 end
8
9 disp(rolls);
```

While Loop Better Example

How many times do we need to roll a die before getting a 6?

```
1 rolls = 0;
2 result = 0;
3
4 while result ~= 6
5     result = randi(6); % generates random integer between 1 and 6
6     rolls = rolls + 1;
7 end
8
9 disp(rolls);
```

We use a `while` loop because we do not know in advance how many rolls will be needed. The loop stops as soon as `result == 6`.

Nested Loops

What if we wanted to find the average number of rolls needed to get a 6?

Nested Loops

What if we wanted to find the average number of rolls needed to get a 6?

```
1 num_trials = 10000;
2 roll_counts = zeros(1, num_trials);
3
4 for trial = 1:num_trials
5     result = 0;
6     rolls = 0;
7
8     while result ~= 6
9         result = randi(6);
10        rolls = rolls + 1;
11    end
12    roll_counts(trial) = rolls;
13 end
14
15 average_rolls = mean(roll_counts);
16 disp(average_rolls);
```

MATLAB script files

MATLAB files (with extension `.m`) can be either *script files* or *function files*.

MATLAB script files

MATLAB files (with extension `.m`) can be either *script files* or *function files*.

A script is simply a sequence of MATLAB commands saved in a file:

- Running a script is essentially equivalent to entering each line of the script into the Command Window yourself, one line at a time.

MATLAB script files

MATLAB files (with extension `.m`) can be either *script files* or *function files*.

A script is simply a sequence of MATLAB commands saved in a file:

- Running a script is essentially equivalent to entering each line of the script into the Command Window yourself, one line at a time.
- The demos we have looked at so far have been scripts.

MATLAB script files

MATLAB files (with extension `.m`) can be either *script files* or *function files*.

A script is simply a sequence of MATLAB commands saved in a file:

- Running a script is essentially equivalent to entering each line of the script into the Command Window yourself, one line at a time.
- The demos we have looked at so far have been scripts.
- Scripts can display text in the Command Window, create figures, and create or modify variables in the workspace.

MATLAB function files

A function file is a reusable piece of code that performs a particular task.

MATLAB function files

A function file is a reusable piece of code that performs a particular task.

- Functions can accept input values and return output values.

MATLAB function files

A function file is a reusable piece of code that performs a particular task.

- Functions can accept input values and return output values.
- Variables created inside a function are only available within that function.

MATLAB function files

A function file is a reusable piece of code that performs a particular task.

- Functions can accept input values and return output values.
- Variables created inside a function are only available within that function.

```
1 function [out1, out2] = functionName(in1, in2, in3)
2     % 1-line comment to describe the function
3     % additional description of the function if needed
4
5     executable code that can use the values in1, in2, ...
6     and at some point assigns values to output parameters out1, out2, ...
7
8 end
```

Saving MATLAB function files

```
1 function [out1, out2] = functionName(in1, in2, in3)
2     % 1-line comment to describe the function
3     % additional description of the function if needed
4
5     executable code that can use the values in1, in2, ...
6     and at some point assigns values to output parameters out1, out2, ...
7
8 end
```

This function needs to be saved in file named `functionName.m`. The file must be in MATLAB's current folder or on the MATLAB path.

Then you can call `functionName()` in other script and function files.

An example function

```
1 function numElmntsOverX = countElmntsOverX(numbers, x)
2     % returns number of elements in numbers greater than x
3
4     % What goes here?
5
6 end
```

An example function

```
1 function numElmntsOverX = countElmntsOverX(numbers, x)
2     % returns number of elements in numbers greater than x
3
4     % What goes here?
5
6 end
```

Once we have this function written, we can do things like:

```
1 disp(countElmntsOverX([3 7 -2 3.2 9], pi)) % should print 3
```

An example function

```
1 function numElmntsOverX = countElmntsOverX(numbers, x)
2     % returns number of elements in numbers greater than x
3
4     % What goes here?
5
6 end
```

Once we have this function written, we can do things like:

```
1 disp(countElmntsOverX([3 7 -2 3.2 9], pi)) % should print 3
```

Activity: write this function yourself and run it.

An example function: first solution

```
1 function numElmntsOverX = countElmntsOverX(numbers, x)
2     % returns number of elements in numbers greater than x
3
4     numElmntsOverX = 0;
5     for num = numbers
6         if num > x
7             numElmntsOverX = numElmntsOverX + 1;
8         end
9     end
10
11 end
```

An example function: first solution

```
1 function numElmntsOverX = countElmntsOverX(numbers, x)
2     % returns number of elements in numbers greater than x
3
4     numElmntsOverX = 0;
5     for num = numbers
6         if num > x
7             numElmntsOverX = numElmntsOverX + 1;
8         end
9     end
10
11 end
```

Can we do this more concisely by removing the `if`?

Hint: MATLAB treats `true` as 1 and `false` as 0.

An example function: second solution

```
1 function numElmntsOverX = countElmntsOverX(numbers, x)
2     % returns number of elements in numbers greater than x
3
4     numElmntsOverX = 0;
5     for num = numbers
6         numElmntsOverX = numElmntsOverX + (num > x);
7     end
8
9 end
```

An example function: second solution

```
1 function numElmntsOverX = countElmntsOverX(numbers, x)
2     % returns number of elements in numbers greater than x
3
4     numElmntsOverX = 0;
5     for num = numbers
6         numElmntsOverX = numElmntsOverX + (num > x);
7     end
8
9 end
```

Can we make this even more concise and remove the loop?

Comparison operations with arrays

Comparison operators apply to arrays element by element.

```
1 numbers = [2 7 4 9 1];  
2 otherNumbers = [3 8 0 0 10];  
3  
4 numbers < otherNumbers;  
5 % [1 1 0 0 1]
```

Comparison operations with arrays

Comparison operators apply to arrays element by element.

```
1 numbers = [2 7 4 9 1];  
2 otherNumbers = [3 8 0 0 10];  
3  
4 numbers < otherNumbers;  
5 % [1 1 0 0 1]
```

Comparing an array to a value compares each element to that value.

```
1 numbers > 3;  
2 % [0 1 1 1 0]
```

Logical operations with arrays

The operators `&` (and), `|` (or), and `~` (not) act on each element of an array.

```
1 a = [true false true];  
2 b = [true true false];  
3  
4 a & b % [1 0 0]  
5 a | b % [1 1 1]  
6 ~a % [0 1 0]
```

Logical operations with arrays

The operators `&` (and), `|` (or), and `~` (not) act on each element of an array.

```
1 a = [true false true];  
2 b = [true true false];  
3  
4 a & b % [1 0 0]  
5 a | b % [1 1 1]  
6 ~a % [0 1 0]
```

They are useful for combining comparisons:

```
1 x = [-2 3 8 12];  
2  
3 (x > 0) & (x < 10)  
4 % [0 1 1 0]
```


An example function: second solution

```
1 function numElmntsOverX = countElmntsOverX(numbers, x)
2     % returns number of elements in numbers greater than x
3
4     numElmntsOverX = 0;
5     for num = numbers
6         numElmntsOverX = numElmntsOverX + (num > x);
7     end
8
9 end
```

Can we make this even more concise and remove the loop?

Hint: We will need what we just learned and the built-in function `sum()`.

An example function: simplest solution

```
1 function numElmntsOverX = countElmntsOverX(numbers, x)
2     % returns number of elements in numbers greater than x
3
4     numElmntsOverX = sum(numbers > x);
5
6 end
```

An example function: simplest solution

```
1 function numElmntsOverX = countElmntsOverX(numbers, x)
2     % returns number of elements in numbers greater than x
3
4     numElmntsOverX = sum(numbers > x);
5
6 end
```

Note that “Concise code” need not be “Good code” in general.

In this case the concise code is easier to read, understand, and check for errors.

More dice rolling

Activity: write the following function

```
1 function numRolls = numRollsForFaces(faceArray)
2 % Roll a d6 until every face in faceArray has been rolled.
3 % numRolls is the number of rolls required.
4
5 % Fill this in!
6 end
```

More dice rolling

Activity: write the following function

```
1 function numRolls = numRollsForFaces(faceArray)
2 % Roll a d6 until every face in faceArray has been rolled.
3 % numRolls is the number of rolls required.
4
5 % Fill this in!
6 end
```

Potentially helpful things:

- `false(size(faceArray))` creates an array containing only `false`, with the same size as `faceArray`.
- `all(A)` returns `true` if every element of the vector `A` is `true`, and `false` otherwise.

More dice rolling

Here is one solution.

```
1 function numRolls = numRollsForFaces(faceArray)
2   % Roll a d6 until every face in faceArray has been rolled.
3   % numRolls is the number of rolls required.
4
5   numRolls = 0;
6   facesRolled = false(size(faceArray));
7
8   while ~all(facesRolled)
9       roll = randi(6);
10      numRolls = numRolls + 1;
11      facesRolled = facesRolled | (faceArray == roll);
12  end
13 end
```